



LMD- Classe S5 - Informatique 2024-2025
Licence L3 + 3^{ing} informatique

TD N° 2 : primitives système de gestion des processus

Questions de cours

Q1 : Dans le système Unix, est ce que tout processus à un père ? Que se passe-t-il lorsqu'un processus devient **orphelin** (mort de son père) ? Quand un processus passe à l'état **zombie** ?

Q2 : Expliquer pourquoi dans Unix, lorsqu'un processus exécute l'appel système **exit ()**, ses ressources ne sont pas libérées tout de suite.

Q3 : quelle est la relation qui existe entre primitive système wait et exit. Comment un processus père attendre ces threads ?

Q4 : Indiquez pourquoi il est important de faire des appels aux fonctions wait dans les processus parents.

Q5 : Qui suis-je ?

- a) Je suis un processus qui s'est terminé mais son père n'a pas encore lu son code de retour.
- b) Je suis un processus dont le père s'est terminé avant lui.
- c) Je suis un appel système qui permet de créer un processus.
- d) Je suis un appel système qui affiche le PID du processus père.

Q6 : Combien de processus fils crée par ces 4 codes programmes :

```
fork() || fork()
fork() && fork()
fork() || (fork() && fork() )
fork() && (fork() || fork() )
```

Exercice N°1

Utiliser le compilateur c sous linux GCC pour compiler les différents programmes tel que la syntaxe est comme suite :

```
#gcc -c nom-fichier.c
#gcc -o nom-executable nom-fichier.o
#./nom-executable
```

L'exécution de programme est comme suite : ./nom-fichier
Pour les threads dans la phase édition ajouter l'option : lpthread
Exemple : #gcc -o nom-executable nom-fichier.o -lpthread

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{

    inti;
    i=0;
    int id;

    if ((id=fork())==0)
    {
        i=2;
        printf("%d\n",i);
        exit(i);
        fork();
        fork();
        i=1;
    }

    wait(NULL);
    printf("%d\n",i) ;
    i=4;
    printf("%d\n",i);
    return 0;
}
```

Q1 : Quel est le résultat affiché par ce programme ?

Q2 : Que se passe-t-il lorsque on supprime l'instruction `exit(i)` ; quelle est le nouveau résultat affiché par ce programme ?

Exercice N°2

Considérez le programme suivant :

```
void main ()
{
    int pid = fork() ;
    x = 0;
    if (pid !=0) { x =x+ 2; }
    else{pid = fork();
        x =x+4;
        if (pid==0) x=x+ 4;
    }
    printf(“%d\n”, x);
}
```

Q1 : Donner le nombre de processus crée dans ce programme, dessiner l’arborescence et indiqué la valeur de x affiché par chaque processus

Exercice N°3

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <stdlib.h>
int i =2;
int main()
{
    pid_t p1,p2;
    if (( p1=fork())==0) f1();
    if ( p1==0) f2();
    else if ((p2= fork())==0) f3();
    if (p1==0) f4();
    if (p2==0) f4();
    pere();
}
```

```

void pere(void)
{
sleep(3);
while ( wait(NULL)> 0);

printf("pere\n");
}
void f1(void)
{
sleep(6);

printf("F1\n");
exit(0);
}
void f2(void)
{ sleep(4);

printf("F2\n");
}

void f3(void)
{ sleep(2);

printf("F3\n");
exit(0);
}
void f4(void)
{ sleep(1);

printf("F4\n");
}

```

- Q1** : indiquez le nombre de processus créés dans ce programme (dessiner l'arborescence)
Q2 : quelle est le nouveau résultat affiché par ce programme si on supprime **else** souligné ?
Q3 : si en supprime exit(0) a la fin de la fonction f1() et f3() , quelle est le résultat affiché ?
Q4 : si en supprime exit de la fonction f1 () et f2 () avec else, quelle est le nouveau résultat ?

Exercice N°4

Q2 : Donnez l'arbre des processus créés, et l'affichage de chacun des deux codes suivant

```

int main()
{
if (fork()==0)
{
printf("S");fflush(stdout);
if (fork()==0)
{printf("SY");}
exit(0);}
wait(0);
wait(0);
}

```

```

if (fork()==0)
{
wait(0);printf("ME");fflush(stdout);
if (fork()==0)
{printf("E");}
exit(0);
}
printf("T");fflush(stdout);

}

```

Exercice N° 5

Vous avez le programme suivant en c :

```

#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>
#include<stdlib.h>
Void main()

{

int j =3, id, i ;
for (i=0 ; i<2; i++)
{
    if ((id=fork())= =0)) {
        j=j-1;
        printf("j =%d\n",j);
        if ((id=fork())==0)) { j = j-1;
            printf("j =%d\n",j);
        }
        exit(0);
    }
}
wait(0);
j = j +1;
printf("j=    %d\n",j);
}

```

Q1 : En supposant que fork() ne renvoie jamais -1, combien de processus vont être créés ? Justifier votre réponse.

Q2 : Quelle est la valeur de j affichée par chacun des processus créés ? Justifier votre réponse.